Task 5 — Comment & Rating System

Overview

Build a review feature where users leave a 1–5 star rating and optional comment on an item, with one editable review per user and a live-aggregated average. This task exercises write + aggregate-read paths, business-rule enforcement, and ordering.

User Story

> ***As a*** *logged-in user,* ***I want*** *to leave a star rating and comment on an item,* ***so that*** *I can share feedback and see the average rating from others.*

Your Story Point Estimation (To Be Completed by Learner)

Estimate the effort for this user story using a Fibonacci scale (1, 2, 3, 5, 8, 13, ...). Consider complexity, effort, risk, and uncertainty, then justify your reasoning.

Factor	Your Assessment
Complexity *(how hard is the logic?)*	*(fill in)*
Effort *(how much work / how many parts?)*	*(fill in)*
Uncertainty / Risk *(unknowns, dependencies?)*	*(fill in)*
Your Estimate (points)	*(fill in)*
Rationale	*(explain how you arrived at your estimate)*

Acceptance Criteria

- A rating is an integer from **1 to 5**; comment text is optional.
- Each user can submit **only one review per item**; submitting again **edits** the existing one.
- The item displays an **average rating** (rounded, e.g., 1 decimal) and total review count.
- Reviews are listed **newest-first**, showing author, stars, comment, and timestamp.
- Only the review's author may edit their review.
- An item with no reviews reports an average of `0` and a count of `0`.

Data Model

Field	Type	Constraints
`id`	UUID / int	Primary key
`itemId`	reference	Indexed
`userId`	reference	**Unique per** `itemId` (one review per user)
`rating`	integer	1–5 inclusive
`comment`	string	Optional
`createdAt` / `updatedAt`	timestamp	Ordering + edit tracking

API Contract (Framework Agnostic)

Method	Endpoint	Purpose	Success	Errors
`POST`	`/items/:id/reviews`	Create/replace own review	`201` / `200`	`400`, `401`
`GET`	`/items/:id/reviews`	List reviews + aggregate	`200`	—
`PUT`	`/reviews/:id`	Edit own review	`200`	`400`, `403`, `404`

`GET /items/:id/reviews` success `200`:

```
{
  "averageRating": 4.0,
  "reviewCount": 3,
  "reviews": [
    { "id": "...", "userId": "...", "rating": 5, "comment": "Great!", "createdAt":
"..." }
  ]
}
```

Back-End Requirements

- Enforce the 1–5 rating range and reject values outside it (`400`).
- Enforce **one review per user per item** via a unique constraint; re-submission updates.
- Compute and return the average rating and review count (recomputed on each change).
- Restrict edits to the authoring user (`403` otherwise).
- Sort returned reviews newest-first.

Front-End Requirements

- Star-rating input (1–5) plus an optional comment box.
- If the user already reviewed the item, pre-fill their review for editing.
- Review list sorted newest-first showing author, stars, comment, and relative time.
- Prominent average-rating badge and total review count.
- Empty state ("Be the first to review") when there are no reviews.

Non-Functional

- Average recalculation stays correct as reviews are added/edited.
- Guard against duplicate submissions (disable submit while in flight).

Edge Cases to Handle

- Rating of 0, 6, or a non-integer → rejected.

- The same user submitting twice → second submission edits, not duplicates.
- Editing another user's review → denied.
- Average of an empty set → `0` (no divide-by-zero).

Unit Test Scenarios (Given / When / Then)

#	Given	When	Then
1	A rating of 0 or 6	Create review	Validation rejects with `400`
2	A user with an existing review	Submit again	Existing review is updated, not duplicated
3	Ratings `[5, 3, 4]`	Aggregate computed	Average = `4.0`, count = 3
4	An item with no reviews	Get reviews	Average = `0`, count = `0`
5	User B editing user A's review	Edit is called	Access denied (`403`)
6	Several reviews over time	List reviews	Ordered newest-first

Deliverables

- Back-end review endpoints with rating validation, one-per-user rule, and aggregation.
- Front-end rating/comment UI with edit support and average display.
- Unit tests covering all scenarios above.
- Short README: how to run the app and tests, plus your completed estimation table.